

Exercise 1: Implementing the Singleton Pattern

CODE :

LOGGER.JAVA :

```
package singleton;

public class Logger {
    private static Logger instance;
    private Logger(){
        System.out.println("object is created");
    }
    public static Logger getInstance() {
        if(instance == null) {
            instance = new Logger();
        }
        return instance;
    }
}
```

MAIN.JAVA :

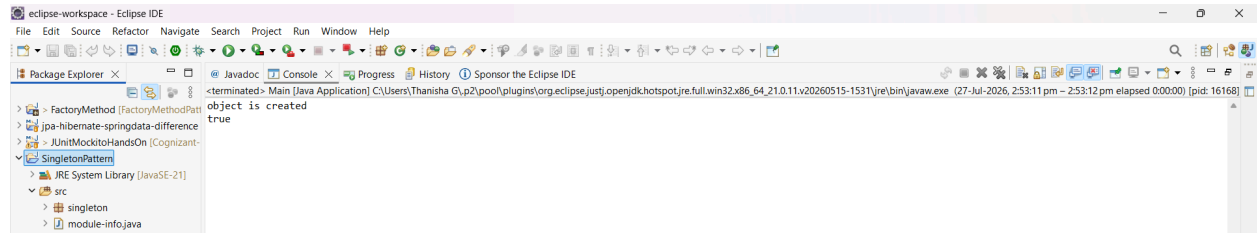
```
package singleton;

public class Main {

    public static void main(String[] args) {

        Logger log1 = Logger.getInstance();
        Logger log2 = Logger.getInstance();
        System.out.println(log1==log2);
    }
}
```

OUTPUT :



Exercise 2: Implementing the Factory Method Pattern

EmailFactory.java :

```
package factory;
```

```
public class EmailFactory implements Notification{
```

```
    @Override
```

```
    public void send() {
```

```
        System.out.println(
```

```
            "Sending Email Notification");
```

```
    }
```

```
}
```

EmailNotification.java :

```
package factory;
```

```
public class EmailNotification
```

```
    implements Notification {
```

```
    @Override
```

```
    public void send() {
```

```
        System.out.println(
```

```
            "Sending Email Notification");
```

```
    }
```

```
}
```

SMSFACTORY.JAVA :

```
package factory;

public class SMSFactory
    extends NotificationFactory {

    @Override
    public Notification createNotification() {

        return new SMSNotification();

    }
}
```

SMSNOTIFICATION .JAVA

```
package factory;

public class SMSNotification
    implements Notification {

    @Override
    public void send() {

        System.out.println(
            "Sending SMS Notification");

    }

}
```

MAIN.JAVA

```
package factory;

public class Main {

    public static void main(String[] args) {
        NotificationFactory factory = new SMSFactory();
        Notification notification = factory.createNotification();
        notification.send();

    }

}
```

}

OUTPUT :

